

economy.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "types.h"
#include "functions.h"

void initEconomy(GameState game[])
{
    game[0].economy.inflationRate = 0;
    game[0].economy.loanInterestRate = LOAN_INTEREST_RATE;

    game[0].economy.constructionSuspendedRoundsLeft = 0;

    game[0].economy.closedPropertyIndex = -1;
    game[0].economy.closedPropertyRoundsLeft = 0;

    game[0].economy.incomeTaxRate = INCOME_TAX_RATE;

    game[0].economy.antiSpeculationActive = 0;

    game[0].economy.currentCardIndex = 0;

    game[0].economy.modifierCount = 0;
}

/* Rule-LK 14: New = Old x (1 + rate/100), never below 1. */
int applyRate(int oldValue, int ratePercent)
{
    int newValue;

    newValue = oldValue + (oldValue * ratePercent) / 100;

    if(newValue < 1)
        newValue = 1;

    return newValue;
}

/* Adds a timed modifier (Rule-LK 30-35). group/index are -1 when not
   relevant. Effects accumulate: two active +25% rents give
   roughly +56%, so simultaneous effects are cumulative (Rule-LK 34). */
void addSourcedModifier(GameState game[], ModifierType type, int group,
                        int index, int percent, int roundsLeft,
                        ModifierSource source)
{
    if(game[0].economy.modifierCount >= MAX_MODIFIERS)
        return;

    game[0].economy.modifiers[game[0].economy.modifierCount].type = type;
    game[0].economy.modifiers[game[0].economy.modifierCount].group = group;
    game[0].economy.modifiers[game[0].economy.modifierCount].index = index;
    game[0].economy.modifiers[game[0].economy.modifierCount].percent = percent;
    game[0].economy.modifiers[game[0].economy.modifierCount].roundsLeft = roundsLeft;
    game[0].economy.modifiers[game[0].economy.modifierCount].source = source;

    game[0].economy.modifierCount++;
}

/* Events, national cards and regulations use this - they are all
   generic effects, not Market Boom/Decline or Regional cards. */
```

```

void addModifier(GameState game[], ModifierType type, int group, int index,
                int percent, int roundsLeft)
{
    addSourcedModifier(game, type, group, index, percent, roundsLeft, SRC_GENERAL);
}

/* Effective percentage multiplier from every active matching modifier,
   multiplied together (Rule-LK 34). Returns 100 when none match.
   A modifier with group == -1 is global and applies to every group,
   so it matches regardless of the requested group (and likewise for
   index == -1). */
int modifierMultiplier(GameState game[], ModifierType type, int group, int index)
{
    int mult;
    int i;

    mult = 100;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != -1 &&
            game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != -1 &&
            game[0].economy.modifiers[i].index != index)
            continue;

        mult = (mult * game[0].economy.modifiers[i].percent) / 100;
    }

    return mult;
}

/* Whether at least one matching modifier is currently active - used
   for flag-style modifiers such as flood risk or recession. */
int isModifierActive(GameState game[], ModifierType type, int group, int index)
{
    int i;

    for(i = 0; i < game[0].economy.modifierCount; i++)
    {
        if(game[0].economy.modifiers[i].type != type)
            continue;
        if(group != -1 && game[0].economy.modifiers[i].group != group)
            continue;
        if(index != -1 && game[0].economy.modifiers[i].index != index)
            continue;

        return 1;
    }

    return 0;
}

/* The effective tax rate used by the Income Tax and Community
   Development Fund squares. Starts at the given base rate (15% /
   10%) and is scaled by the currently active global value
   modifiers, so market fluctuations move the rate up or down. */
int currentTaxRatePercent(GameState game[], int baseRate)
{
    return (baseRate * modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1)) / 100;
}

```

```

}

/* Ticks every active modifier down and removes the expired ones. */
void decrementModifiers(GameState game[])
{
    int i;

    i = 0;

    while(i < game[0].economy.modifierCount)
    {
        game[0].economy.modifiers[i].roundsLeft--;

        if(game[0].economy.modifiers[i].roundsLeft <= 0)
        {
            int j;

            for(j = i; j < game[0].economy.modifierCount - 1; j++)
                game[0].economy.modifiers[j] = game[0].economy.modifiers[j + 1];

            game[0].economy.modifierCount--;
        }
        else
        {
            i++;
        }
    }
}

```

```

/* Formats an amount with thousands separators, e.g. 30000 -> "30,000". */
void formatLKR(int amount, char out[])
{
    char digits[32];
    int len;
    int i;
    int w;
    int neg;

    neg = 0;

    if(amount < 0)
    {
        neg = 1;
        amount = -amount;
    }

    sprintf(digits, "%d", amount);
    len = strlen(digits);

    w = 0;

    if(neg)
        out[w++] = '-';

    for(i = 0; i < len; i++)
    {
        if(i > 0 && (len - i) % 3 == 0)
            out[w++] = ',';

        out[w++] = digits[i];
    }

    out[w] = '\0';
}

```

```

}

/* Every 10 rounds, a random inflation rate is applied to every
   property's price, rent, and construction costs. */
void applyInflation(GameState game[])
{
    int possibleRates[6] = {-3, 0, 2, 5, 8, 12};
    int rate;
    int i;

    rate = possibleRates[rand() % 6];
    game[0].economy.inflationRate = rate;

    printf("\n=== Inflation Update ===\n");
    printf("New Inflation Rate : %d%%\n", rate);

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        game[0].board[i].property.purchasePrice =
            applyRate(game[0].board[i].property.purchasePrice, rate);

        game[0].board[i].property.mortgageValue =
            applyRate(game[0].board[i].property.mortgageValue, rate);

        if(game[0].board[i].type == PROPERTY)
        {
            game[0].board[i].property.baseRent =
                applyRate(game[0].board[i].property.baseRent, rate);

            game[0].board[i].property.houseCost =
                applyRate(game[0].board[i].property.houseCost, rate);

            game[0].board[i].property.hotelCost =
                applyRate(game[0].board[i].property.hotelCost, rate);
        }
    }

    game[0].economy.loanInterestRate = applyRate(game[0].economy.loanInterestRate, rate);

    printf("New Loan Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
}

/* A property's real current worth: purchase price, minus age
   depreciation, adjusted by any active market conditions. Reused
   everywhere a "real price" is needed - buying, insurance, repairs,
   renovation, net worth - so all of them stay consistent. */
/* Rule-LK 11-13, 24: the current market value of a property, after
   age depreciation and every value modifier currently active
   (Rule-LK 34 - all matching modifiers multiply together). */
int currentMarketValue(GameState game[], int propIndex)
{
    int price;
    int depreciation;
    int mult;

    price = game[0].board[propIndex].property.purchasePrice;

```

```

depreciation = game[0].board[propIndex].property.depreciation;

price = price - (price * depreciation) / 100;

mult = modifierMultiplier(game, MOD_VALUE_GLOBAL, -1, -1);

if(game[0].board[propIndex].type == PROPERTY)
    mult = (mult * modifierMultiplier(game, MOD_GROUP_VALUE,
                                      game[0].board[propIndex].property.group, -1)) / 100;

if(game[0].board[propIndex].type == RAILWAY)
    mult = (mult * modifierMultiplier(game, MOD_RAIL_VALUE, -1, -1)) / 100;

mult = (mult * modifierMultiplier(game, MOD_INDEX_VALUE, -1, propIndex)) / 100;

price = (price * mult) / 100;

return price;
}

/* Rule-LK 15, 16: properties get older every round, and lose value
   past age 50. Prints the required "Property Depreciation" message
   (Section 5) whenever a property's depreciation actually changes. */
void ageProperties(GameState game[])
{
    int i;
    int roundsOver50;
    int newDepreciation;
    int oldDepreciation;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        game[0].board[i].property.age++;

        if(game[0].board[i].property.age > PROPERTY_AGE_LIMIT)
        {
            oldDepreciation = game[0].board[i].property.depreciation;

            roundsOver50 = game[0].board[i].property.age - PROPERTY_AGE_LIMIT;
            newDepreciation = roundsOver50 / DEPRECIATION_STEP;

            if(newDepreciation > MAX_DEPRECIATION)
                newDepreciation = MAX_DEPRECIATION;

            game[0].board[i].property.depreciation = newDepreciation;

            if(newDepreciation != oldDepreciation)
            {
                printf("\nProperty Depreciation\n");
                printf("Property\n%s\n", game[0].board[i].name);
                printf("has depreciated by %d%%.\n", newDepreciation);
                printf("Current Value\nLKR %d\n", currentMarketValue(game, i));
            }
        }
    }
}

/* Rule-LK 17: landing on your own aged property offers a renovation. */
void tryRenovateAgeDepreciation(GameState game[], int playerIndex, int propIndex)
{

```

```

int cost;

if(game[0].board[propIndex].property.depreciation <= 0)
    return;

if(!shouldRenovateAgeDepreciation(game, playerIndex,
                                   game[0].board[propIndex].property.depreciation))
{
    return;
}

cost = (currentMarketValue(game, propIndex) * RENOVATION_COST_PERCENT) / 100;

if(game[0].players[playerIndex].cash < cost)
    return;

payMoney(game, playerIndex, cost);

game[0].board[propIndex].property.age = 0;
game[0].board[propIndex].property.depreciation = 0;

game[0].board[propIndex].property.baseRent =
    applyRate(game[0].board[propIndex].property.baseRent, RENOVATION_RENT_BOOST);

printf("\n%s renovated %s.\n",
        game[0].players[playerIndex].name, game[0].board[propIndex].name);

printf("Renovation Cost : LKR %d\n", cost);
}

/* Table 3: rent collected as a percentage of building condition. */
int rentConditionPercent(int condition)
{
    if(condition >= 90)
        return 100;
    if(condition >= 75)
        return 90;
    if(condition >= 50)
        return 75;
    if(condition >= 25)
        return 50;

    return 0; /* below 25% - building closed */
}

/* Rule-LK 25, 28: building condition wears down every round, and
   20+ rounds of neglect causes lasting structural damage. */
void ageBuildings(GameState game[])
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        /* Business Interruption's 5-round lost-income countdown runs
           even if the property isn't currently developed with houses
           (it still has a hotel that just isn't earning anything). */
        if(game[0].board[i].property.lostIncomeRoundsLeft > 0)
            game[0].board[i].property.lostIncomeRoundsLeft--;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)

```

```

        continue;

game[0].board[i].property.condition -= 2;

if(game[0].board[i].property.condition < 0)
    game[0].board[i].property.condition = 0;

game[0].board[i].property.roundsSinceMaintenance++;

if(game[0].board[i].property.roundsSinceMaintenance > NEGLECT_ROUNDS &&
    !game[0].board[i].property.structurallyDamaged)
{
    game[0].board[i].property.structurallyDamaged = 1;

    game[0].board[i].property.preDamagePurchasePrice =
        game[0].board[i].property.purchasePrice;
    game[0].board[i].property.preDamageBaseRent =
        game[0].board[i].property.baseRent;

    game[0].board[i].property.purchasePrice =
        game[0].board[i].property.purchasePrice -
        (game[0].board[i].property.purchasePrice * STRUCTURAL_VALUE_LOSS) / 100;

    game[0].board[i].property.baseRent =
        game[0].board[i].property.baseRent -
        (game[0].board[i].property.baseRent * STRUCTURAL_RENT_LOSS) / 100;

    game[0].board[i].property.maintenanceCostMultiplierPercent = DAMAGE_REPAIR_MULTIPLIER;
    game[0].board[i].property.condition = 0;

    printf("\n%s has suffered structural damage from neglect!\n",
        game[0].board[i].name);
}
}

/* Rule-LK 27: restores a building's condition to 100% for a fee. */
void performMaintenance(GameState game[], int playerIndex)
{
    int i;
    int baseCost;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.houses == 0 && !game[0].board[i].property.hotel)
            continue;

        if(game[0].board[i].property.structurallyDamaged)
            continue; /* needs a full renovation instead */

        if(game[0].board[i].property.condition >= 100)
            continue;

        if(game[0].board[i].property.hotel)
            baseCost = (game[0].board[i].property.hotelCost * 8) / 100;
        else

```

```

        baseCost = (game[0].board[i].property.houseCost * 5) / 100;

        cost = (baseCost * game[0].board[i].property.maintenanceCostMultiplierPercent) / 100;

        if(!shouldMaintain(game, playerIndex, game[0].board[i].property.condition, cost))
            continue;

        if(game[0].players[playerIndex].cash < cost)
            continue;

        payMoney(game, playerIndex, cost);

        game[0].board[i].property.condition = 100;
        game[0].board[i].property.roundsSinceMaintenance = 0;

        printf("\n%s performed maintenance on %s.\n",
            game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Maintenance Cost : LKR %d\n", cost);
    }
}

/* Rule-LK 29: fully restores a structurally damaged building. */
void renovateStructuralDamage(GameState game[], int playerIndex)
{
    int i;
    int replacementValue;
    int cost;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY)
            continue;

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.structurallyDamaged)
            continue;

        if(game[0].board[i].property.hotel)
            replacementValue = game[0].board[i].property.hotelCost;
        else
            replacementValue = game[0].board[i].property.houseCost;

        cost = (replacementValue * DAMAGE_REPAIR_MULTIPLIER) / 100;

        if(game[0].players[playerIndex].cash < cost)
            continue;

        payMoney(game, playerIndex, cost);

        game[0].board[i].property.purchasePrice = game[0].board[i].property.preDamagePurchasePrice;
        game[0].board[i].property.baseRent = game[0].board[i].property.preDamageBaseRent;
        game[0].board[i].property.condition = 100;
        game[0].board[i].property.structurallyDamaged = 0;
        game[0].board[i].property.maintenanceCostMultiplierPercent = 100;
        game[0].board[i].property.roundsSinceMaintenance = 0;

        printf("\n%s fully renovated %s after structural damage.\n",
            game[0].players[playerIndex].name, game[0].board[i].name);

        printf("Renovation Cost : LKR %d\n", cost);
    }
}

```

